Bitcoin Double-Spend Website

CSCE A470 Final Report

Derek Hodgkins, Caden Kane, Lars Goozen, Zong Xiong

4/30/2026

Table of Contents

Abstract 1
1. Introduction 1
2. Project Overview 2
    2.1 Prior Tools and Project Motivation 2
    2.2 Planning Process and Use Scenarios 2
    2.3 Parties Involved 3
3. Project Requirements 3
    3.1 Functional Specifications 3
    3.2 System Specifications 4
4. System Design 4
    4.1 User Interface Design 5
    4.2 Data Structures 6
    4.3 System Architecture 7
    4.3.1 Bitcoin Core Node 7
    4.3.2 Python Data Processor 7
    4.3.3 .NET Core API Middleware 8
    4.3.4 Blazor Web Framework 8
    4.4 Data Processing and Probability Workflow 9
    4.4.1 Transaction Retrieval and Normalization 9
    4.4.2 Double-Spend Probability Model 9
    4.4.3 Historical and Live Graph Generation 10
5. Software Development Process 11
    5.1 Testing and Debugging 11
    5.2 Prototyping Challenges 12
    5.3 Code Review Actions 12
6. Results and Discussion 13
    6.1 Final Program 13
    6.2 Limitations 14
    6.3 Future Steps 15
7. Summary and Conclusion 15
8. References 16
Appendix A: User Manual 17
Appendix B: Code Listing 20
Appendix C: Weekly Meeting Logs 22

Bitcoin Double-Spend Website

Derek Hodgkins, Lars Goozen, Zong Xiong, Caden Kane

Abstract

The Bitcoin Double-Spend Website is a web application that combines a block explorer-style transaction summary with a live double-spend risk evaluation. Users enter a Bitcoin transaction ID and the system retrieves transaction data from a Bitcoin Core node, summarizes important details such as inputs, outputs, block status, confirmations, and time, and displays both historical and live probability graphs. The project was created because common blockchain explorers make it easy to inspect transaction details, but they do not normally explain how double-spend risk changes as a transaction waits for confirmations [1], [2]. The final prototype uses a Blazor web interface, a .NET Core API middleware layer, a Python/Uvicorn data processor, and Bitcoin Core node. The result is an educational tool that helps users connect Bitcoin confirmation depth, elapsed time, and attacker hash-power assumptions to the practical risk of accepting a transaction too early [2].

1. Introduction

Bitcoin is designed to let users transfer value without a central authority. A core security problem in such a system is the double-spend problem: an attacker may try to spend the same coins in one transaction while secretly or publicly attempting to replace that transaction with another. Bitcoin reduces this risk through proof-of-work, block confirmations, and the steep economic cost of creating an alternate chain. However, transaction acceptance is still probabilistic rather than instantaneous. A payment with zero confirmations can be convenient, but it is less secure than a payment buried under several blocks.

Many public blockchain explorers, such as Blockchain.com and Blockchair.com, are excellent for looking up raw transaction details, block information, addresses, and confirmations. Their purpose is primarily inspection. They generally do not act as teaching tools for double-spend acceptance policies, and they do not show a live probability curve for a selected transaction. Neumayer's time-based double-spend model is important to this project because it shows that transaction risk depends on both confirmation count and elapsed time, not confirmations alone [2]. This gap motivated our project. Our goal was not to replace a full blockchain explorer, but to build a focused website that makes double-spend risk easier to understand by tying transaction details to live and historical probability graphs.

The project is also connected to course topics in cryptocurrency technologies. The application uses a real Bitcoin Core node, RPC calls, live block notifications, JSON data transfer, WebSocket streaming, and a risk model derived from Bitcoin double-spend analysis. This lets the

website serve both as a software engineering capstone and as a practical demonstration of blockchain transaction acceptance

2. Project Overview

The finished application accepts a transaction hash and returns a web page containing three categories of information: a transaction summary, historical probability graphs, and a live probability stream. The summary gives the user the basic context needed to interpret the transaction. The graphs then show how the probability of a successful double-spend changes over time and across confirmations. This design was chosen because the user should not need to read raw Bitcoin RPC responses or manually calculate risk from a formula.

At a high level, the user interacts only with the Blazor website. The website calls the .NET middleware API, which then communicates with the Python data processor. The Python processor queries the Bitcoin Core node for raw transaction and block data. Once the data has been normalized, it is returned to the front end as JSON objects that match the front-end data transfer objects. For live updates, the browser opens a WebSocket stream so the probability graph can update without refreshing the page.

2.1 Prior Tools and Project Motivation

Existing blockchain explorers are valuable because they make the Bitcoin ledger searchable. A user can paste in a transaction ID and see whether the transaction is confirmed, which block contains it, and which inputs and outputs are involved. For a course project about blockchain security, that was only part of the story. We wanted the interface to answer a more analytical question: how risky is it to accept this transaction right now?

The project therefore focuses on the relationship between confirmations and the probability that an attacker could catch up. The website is intended to be useful for students, developers, or merchants who need a visual explanation of why confirmation count matters. The program is not designed to make financial recommendations; rather, it presents a model and shows how changing the security assumptions changes the output.

2.2 Planning Process and Use Scenarios

The team used a prototype-driven planning process. Instead of specifying every screen and endpoint at the beginning, we built pieces of functionality, tested them with real Bitcoin transactions, and refined the design when edge cases appeared. The following use scenarios guided the design:

- A user searches for a Bitcoin transaction and reviews its key details, including confirmations, inputs, outputs, value movement, and block information.

- A user searches for a recent or unconfirmed transaction and uses the probability score and graph to evaluate the current double-spend risk.
- A user searches for an older confirmed transaction and expects the system to recognize that its risk is effectively resolved.
- A user searches for different transaction types, such as standard transactions, mempool transactions, or coinbase transactions, and expects the interface to present the available information clearly and accurately.
- A user compares the transaction details and double-spend risk visualization to better understand how confirmation count affects transaction security.

2.3 Parties Involved

The project idea was created by Dr. Sebastian Neumayer who was our contact and advisor for the project throughout the duration of the work. Dr. Neumayer wrote "*An Analysis of Acceptance Policies of Blockchain Transactions"* for which our double-spend probabilities are based on. We met with our contact every Wednesday for guidance on developing our project and best practices for implementing the double-spend probability calculations.

The Bitcoin Core node is hosted on the University's ADAC server, which provides a reliable and secure environment well-suited for the demands of running a full node. This was made possible through the generous support of Dr. Kenrick Mock and Dr. Frank Witmer. They were able to assist and make sure the server was using the latest Linux environments.

3. Project Requirements

The requirements were expressed as functional and system specifications. They changed during development as the team discovered practical issues in Bitcoin transaction data and front-end charting. The final requirements emphasize a usable website, reliable data retrieval, live visualization, and maintainable deployment.

3.1 Functional Specifications

1. The website shall provide a search field where a user can enter a Bitcoin transaction ID.
2. The system shall retrieve transaction details from a Bitcoin Core node rather than relying on a third-party blockchain explorer API.
3. The transaction summary shall display the transaction ID, confirmation count, block hash or mempool status, timestamp when available, input/output addresses/values, and whether the transaction is coinbase or too old.
4. The UI shall display double-spend probability in a readable manner rather than only showing raw numeric data.
5. The system shall generate historical graph data for the first hours after the transaction and for the full confirmation history when data is available.

6. The system shall provide a live graph that updates as elapsed time and confirmation change.
7. The system shall allow model parameters, such as attacker hash-power assumption, to be changed without rewriting the core UI.
8. The system shall handle representative transaction cases, including confirmed transactions, old transactions, mempool transactions, and coinbase transactions
9. The website shall present useful error messages when a transaction cannot be found or when a back-end component is unavailable.
10. The source code and documentation shall be organized so another developer can easily continue development and run the system locally.

3.2 System Specifications

Table 1. System Specifications

Component	**Specification**
Bitcoin node	Bitcoin Core full node with RPC access enabled; ZMQ notification used for real-time block events.
Back-end processor	Python service hosted with Uvicorn/ASGI. It exposes HTTP endpoints for summaries and historical graph data, plus a WebSocket endpoint for live updates.
Middleware	.NET Core API layer that separates the Blazor client from the back end implementation details and maps back-end JSON into typed C# data transfer objects.
Front end	Blazor web UI with ApexCharts-based probability graphs and transaction summary cards.
Deployment target	Linux server environment. Docker/Docker Compose packaging is recommended so the .NET and Python services can run consistently. Bitcoin Core data should remain in a persistent host volume or external node.
Browser support	Modern desktop browsers such as Chrome, Edge, or Firefox. The layout should also remain usable on narrower screens.

4. System Design

The system is divided into four main components: the Blazor web framework, the .NET Core API middleware, the Python data processor, and the Bitcoin Core node. This separation was intentional. The front end can focus on interaction and display, while the back end can focus on blockchain data retrieval, probability calculations, and live updates.

4.1 User Interface Design

The user interface follows a simple search-and-results pattern. When the page first loads, the user sees an empty state explaining what double spending is and that a transaction ID is needed. After entering a valid transaction ID, the result view displays a set of summary cards and probability graphs. This design was chosen because it matches the mental model of a blockchain explorer while adding risk analysis to the result page.

The main UI elements are:

- Search bar: accepts a Bitcoin transaction hash and submits it to the back end.
- Transaction summary cards: display block hash, timestamp, confirmation count, inputs, outputs, and special statuses such as mempool or coinbase.
- Risk summary card: shows the current double-spend probability and a readable risk label for the slider-based user input for the attacker relative hash power variable.
- Historical probability graphs: show how risk changes over the first five-hour window and over the full available history.
- Live probability graph: updates through a WebSocket stream so the user can watch the model change over time.

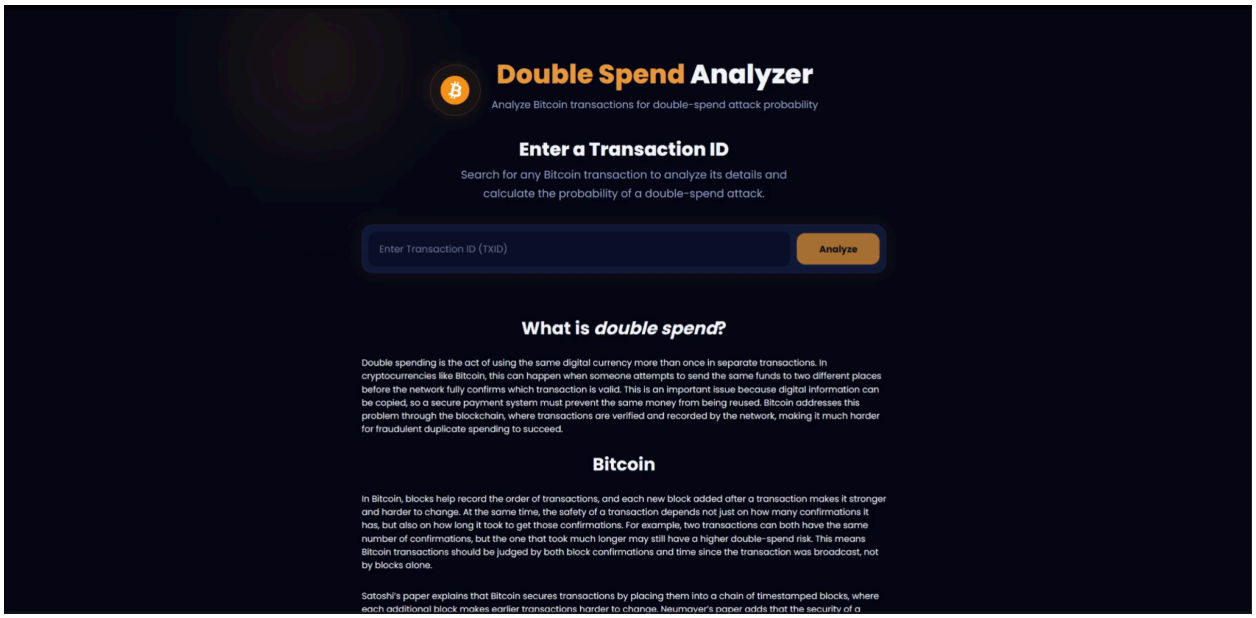


Figure 1. Website Homepage

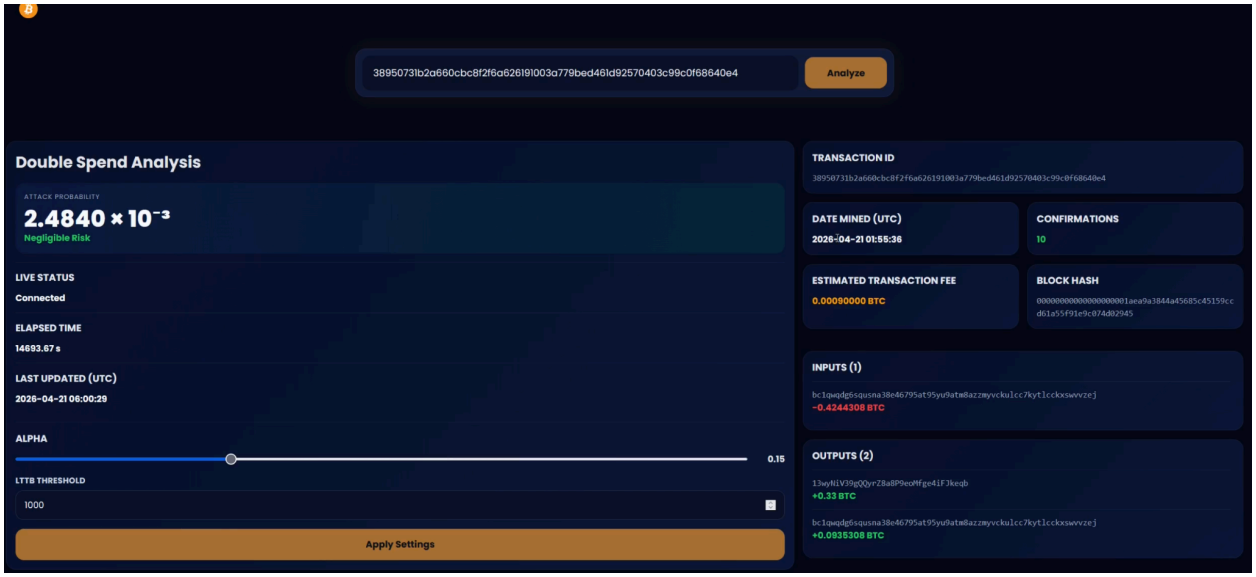


Figure 2. Transaction search result

4.2 Data Structures

The application uses simple data transfer objects rather than exposing raw Bitcoin RPC responses to the UI. The RPC data is large and contains fields that are not relevant to the display. The processor filters and normalizes the data into smaller JSON objects, and the .NET/Blazor side maps those objects into matching C# models.

Table 2. Main Data Structures

Data Structure	Purpose	Important Fields
Transaction summary	Stores the transaction information displayed in summary cards.	txid, confirmations, is_coinbase, blockhash, time, inputs, outputs
Input/output item	Represents the addresses and values that make up the transaction.	address or addresses, value
Probability point	Stores a chart point for historical and live graphs.	elapsed seconds, probability, confirmation index or nullable confirmation number
Transaction store/cache	Keeps already-fetched transaction data available for WebSocket updates.	txid key, summary data, confirmation data, probability state

Front-end chart collections	Hold the points passed into ApexCharts.	five-hour points, full-history points, live points

4.3 System Architecture

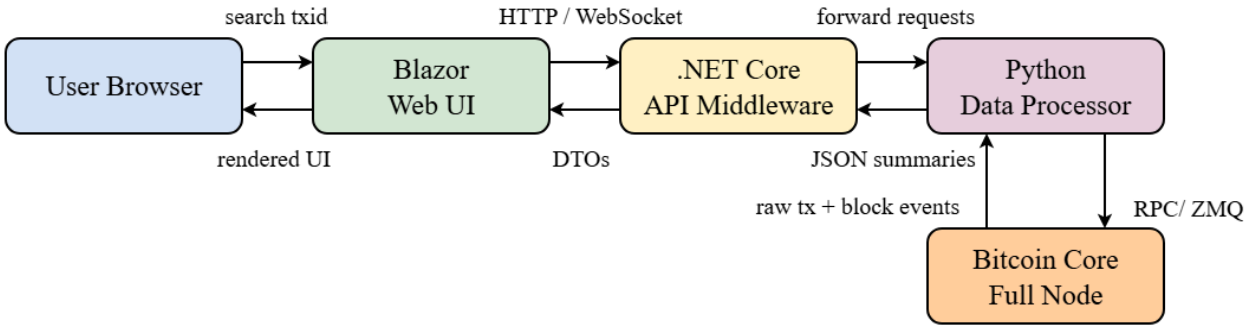


Figure 3. Bitcoin Double-Spend Website System Architecture

4.3.1 Bitcoin Core Node

The primary data source for the website is a Bitcoin Core node hosted on UAA's ADAC server. Bitcoin Core is the official open-source implementation of Bitcoin that turns the host machine into a peer on the Bitcoin peer-to-peer network. A blockchain peer's main responsibility is to maintain an up-to-date copy of the blockchain ledger, validate transactions and propagate information to other peers. This blockchain ledger is the main data source for the project because it is constantly updated with the latest transactions while also maintaining historical blockchain data dating back to Bitcoin's launch in 2009. As of April 2026, there are about 780 GB of transaction data available on the blockchain. Bitcoin Core also includes a built-in ZMQ messaging service, which allows for real-time event notifications and is used in this project to detect when a new block is added to the blockchain.

4.3.2 Python Data Processor

The main component of the back end is the Python data processor, hosted on a Uvicorn ASGI server, which is responsible for fetching and processing data from the Bitcoin Core node and sending it to the front end. The Python ASGI server contains both HTTP and WebSocket endpoints that are used for communication with the front end. The HTTP endpoints are used for static request data such as transaction data and graphs displaying precomputed historical probability data, while the WebSocket endpoint is used for sending dynamic data such as double-spend probabilities and block confirmations. When a user wants information on a transaction, the front end sends a transaction ID to the back-end server's HTTP endpoint. When

the back-end server receives this transaction ID, it initiates a *getrawtransaction* request via RPC to the Bitcoin Core node which sends back the raw transaction data. Most of the data sent from the Bitcoin Core node is not needed, so the back-end server filters the data so that only the required information is kept. The data is then used to build data points to be used in graphs displaying the historical double-spend probability. The transaction data, along with the graphical data points, is packaged into separate JSON objects and sent to the front end through the HTTP endpoints. Next, a WebSocket connection is initiated with the front end to open up a data stream for real-time double-spend probabilities. When the front end initiates this connection, the back end uses the data contained in the already existing transaction JSON to calculate the probability of a double-spend in real time. Every half-second, the back end calculates the probability of a double-spend, builds data points for the live front-end graphs, packages them into JSON, and sends them over the WebSocket connection.

4.3.3 .NET Core API Middleware

The .NET Core middleware API sits between the Blazor UI and the Python data processor. This layer keeps the front end from depending directly on Python endpoint details. Instead of requiring the Blazor components to know the exact structure of the Python responses, the middleware defines shared C# models and translates back-end JSON objects into the shape expected by the UI. For example, transaction summaries, probability graph points, and confirmation updates can be represented as strongly typed data transfer objects before they are displayed by the front end. This layer also provides a natural place to perform response validation and handle errors before they reach the user interface. This separation made debugging easier because front-end display issues could be separated from raw Bitcoin RPC issues and Python processing issues. It also made the project easier to organize because the Blazor UI could focus on presentation while the middleware handled communication with the back end.

4.3.4 Blazor Web Framework

The Blazor web framework provides the user-facing application for the project. It renders the home/search page, transaction summary cards, transaction detail sections, and graph components. After a user submits a transaction ID, the Blazor interface displays the returned transaction information in a readable format, including values such as the block hash, confirmation count, transaction time, inputs, outputs, and risk information. The graph components use ApexCharts to display double-spend probability over time and by confirmation count. These visualizations help the user understand that transaction risk generally decreases as more blocks are confirmed after the transaction.

Blazor components such as the search results view and live probability graph are responsible for calling the .NET API, maintaining local chart point lists, and re-rendering the page as live data

arrives. This component-based structure made it easier to separate the major parts of the interface, such as the search bar, summary cards, and graphs. It also helped during debugging because display logic, API communication, and chart rendering could be adjusted in separate files instead of being mixed into one large page.

4.4 Data Processing and Probability Workflow

The data processing and probability workflow describes how the application takes a client-submitted Bitcoin transaction ID and converts it into the transaction details, probability calculations, and graph data shown on the webpage. This section provides a pathway for the more detailed back-end processes explained in the following subsections. The workflow starts by retrieving transaction data from Bitcoin Core so that transaction information, mempool status, and confirmation data can be handled consistently. The normalized data is then used in the double-spend probability model to estimate the risk of a double-spend attack based on time, confirmations, and the attacker's hash power. Finally, the back end prepares historical and live graph data, including the use of downsampling with the Largest-Triangle-Three-Buckets algorithm, also known as LTTB, so the front end can display the graphs, probability values, and transaction information clearly and efficiently.

4.4.1 Transaction Retrieval and Normalization

1. Receive a transaction ID from the front end.
2. Call Bitcoin Core through RPC to retrieve the raw transaction and verbose metadata.
3. Check special transaction cases, including mempool and coinbase transactions.
4. Resolve input addresses and values when the transaction is not coinbase. For coinbase transactions, mark the input as COINBASE instead of trying to fetch a previous output.
5. Filter the raw response into the fields required by the UI.
6. Return the normalized summary and graph-ready arrays as JSON.

This normalization step was one of the most important parts of the project because raw Bitcoin transactions are not always shaped the same way. A mempool transaction has no confirmed block time. A coinbase transaction does not have ordinary spendable previous inputs. Older transactions may have many confirmations and do not need frequent live polling. Handling these cases in the back end kept the UI logic simpler.

4.4.2 Double-Spend Probability Model

The double-spend model used in this project is built on Satoshi Nakamoto's original Bitcoin double-spend analysis [1], and the model used is from Sebastian Neumayer's research [2] on blockchain transaction acceptance policies. The purpose of the model is to calculate the probability that an attacker can overtake the honest Bitcoin chain and initiate a double-spend attack. Nakamoto's model describes the idea of a double-spend risk that decreases as the honest

chain extends its lead from the attacker's chain. If the honest network controls probability p, the attacker controls probability q, and the honest chain is z blocks ahead, the probability that the attacker can catch up is given by this equation:

$$q_z = \begin{cases} 1, if\ p \leq q \\ (\frac{q}{p})^2, if\ p > q \end{cases}$$

This equation explains why confirmations matter, but it is missing an important variable, time. Time matters for double-spend since Bitcoin blocks do not arrive at fixed ten-minute intervals. As time passes without new honest-chain confirmations, the attacker has more opportunity to mine blocks on a competing chain and potentially overtake the honest chain. Neumayer's model is more accurate since it is a function of time, so it can account for the attacker's mining opportunity while confirmations are delayed. In Neumayer's equation, T represents the time since the transaction was broadcast, N represents the number of main-chain blocks added since the transaction was broadcast, and α represents the attacker's fraction of hash power relative to the network. The time-based probability of a double-spend equation provided by Neumayer, and used for this project is:

$$P_{\text{double_spend_if_accepted_right_now}}(T,N) = \sum_{i=-N}^{0} P_{\text{attacker_ahead_by_}i}(i,T,N)\left(\frac{\alpha}{1-\alpha}\right)^{-i+1} + 1 - \sum_{i=-N}^{0} P_{\text{attacker_ahead_by_}i}(i,T,N)$$

In this equation $P_{\text{attacker ahead by i}}(\underline{i}, T, N)$ represents the probability the attacker is in state i after time T and since N main-chain blocks have extended the chain. As T increases without new honest-chain confirmations, the attacker has more time to mine blocks on a competing chain. Furthermore, a smaller value of T gives the attacker less time, resulting in a lower probability. The first summation provides the probability of the case where the attacker is currently behind and may catch up later. The second summation is the remaining probability where the attacker is already ahead.

In our implementation we utilize the Python back end to make our time-based model calculation for all of our graphs and double-spend calculations. The historical graph uses previous block times, while the live graph updates as time continues and blocks are detected.

4.4.3 Historical and Live Graph Generation

The historical graph algorithm builds arrays of probability points before the page is rendered. The five-hour graph emphasizes the most important part of a transaction's lifetime, while the full-history graph focuses on confirmation history. For a time-based graph, elapsed seconds are

used as the x-axis. For a confirmation-based graph, the confirmation number is used so the line can step when confirmations change. Keeping these two interpretations separate helped avoid misleading graphs.

For historical transactions with larger datasets, the back end applies the Largest-Triangle-Three-Buckets algorithm, also known as LTTB, to downsample the transaction and probability data before it is sent to the front end. LTTB reduces the number of plotted points while preserving the overall shape of the original graph. This helps reduce load time and improves efficiency because the browser does not need to render every calculated point. As a result, the graph remains readable while still accurately representing the original probability trend.

The live graph algorithm uses the WebSocket connection. Each update cycle checks the stored transaction state, recalculates the probability, packages the new value into a probability point, and sends it to the front end. The UI appends the point to the chart without requiring the page to reload. For heavily confirmed transactions, the live stream can slow down or stop because the probability no longer changes in a meaningful way for normal demonstration purposes.

5. Software Development Process

The team used an iterative development process. Early work focused on proving that Bitcoin Core RPC data could be fetched and summarized. Later work focused on turning that data into graphs, connecting the Python back end to the .NET and Blazor front end, and refining edge cases discovered through testing.

5.1 Testing and Debugging

Testing was performed with representative transaction IDs instead of only using ideal examples. This exposed several bugs that would not appear in a simple confirmed transaction. Important debugging areas included:

- Confirmation counts returning zero because the back-end summary was not using the correct block or confirmation fields.
- Old transactions caused server errors because the back end tried to perform live graph logic that was unnecessary for deeply confirmed transactions.
- Coinbase transactions being treated like normal transactions even though they do not have ordinary inputs.
- Mempool transactions displaying an invalid 1969 timestamp because the UI interpreted a missing Unix time as zero.
- Chart x-axes showing too many ticks or using the wrong scale for the first-window graph.
- Step-line confirmation graphs losing their step behavior during UI refactoring.

- Python and C# DTO mismatches, such as a field being computed in the back end but not exposed to the Blazor side.
- WebSocket connection behavior when alpha or other model parameters changed while the live graph was running.

The final testing strategy should include a small regression list of transaction IDs: one recently confirmed transaction, one old deeply confirmed transaction, one mempool transaction if available, and one coinbase transaction. Each should be tested after major back end or UI changes.

5.2 Prototyping Challenges

The largest challenge was integrating several technologies with different responsibilities. Bitcoin Core provides raw blockchain data, Python performs blockchain-specific processing, .NET provides typed middleware, and Blazor renders the interface. A change in one layer often required a matching change in another. For example, adding an is_coinbase field in Python was not useful until the .NET DTO and Blazor UI also understood that field.

Another challenge was deciding how to present probability graphs honestly. Time-based risk and confirmation-based risk are related but not identical visualizations. The team had to refine the graph components so the five-hour view used elapsed time while the confirmation-focused view used confirmation number and step lines. This was a design issue as much as a programming issue because the graph should teach the concept without implying false precision.

Deployment planning also became an important issue near the end of the project. The team discussed using Docker so the Python service and .NET/Blazor service can run consistently on a Linux server. Bitcoin Core is much larger than the other services, so the recommended deployment is to containerize the application layers while treating the full node and blockchain data directory as a persistent external dependency.

5.3 Code Review Actions

Table 3. Code review defects and fixes

Defects	Page	Fixes
Should check if fields exist before accessing them (transaction_data["value"] and transaction["vout"])	routes.py	Added a default value if field does not exist transaction_data.get(“value”,0)
limit elapsed time so it never goes below 0	transaction_probabilities.py	Changed elapsed time equation to max(0,time.time() -

		blocktime)
could remove vin_summary dict and just store its value as a list; maybe combine iterations as well	data_processor.py	Omitted vin_summary and built the vin data store directly instead.
transaction_store is a dict, don't use i as iterator; use 'key' or something else more meaningful	routes.py	Changed TRANSACTION_STORE[i] to TRANSACTION_STORE[txid]

6. Results and Discussion

6.1 Final Program

The final prototype meets the main goal of the project. A user can enter a Bitcoin transaction ID and receive a readable transaction summary along with double-spend probability visualizations. The UI is more specialized than a normal explorer because it focuses on transaction acceptance risk instead of trying to display every possible blockchain field.

The most successful part of the project is the connection between real blockchain data and live visualization. The application does not use static example transactions; it queries Bitcoin Core and builds the display from current node data. This makes the website a stronger educational tool because users can test real transactions and observe how confirmations affect the graph.

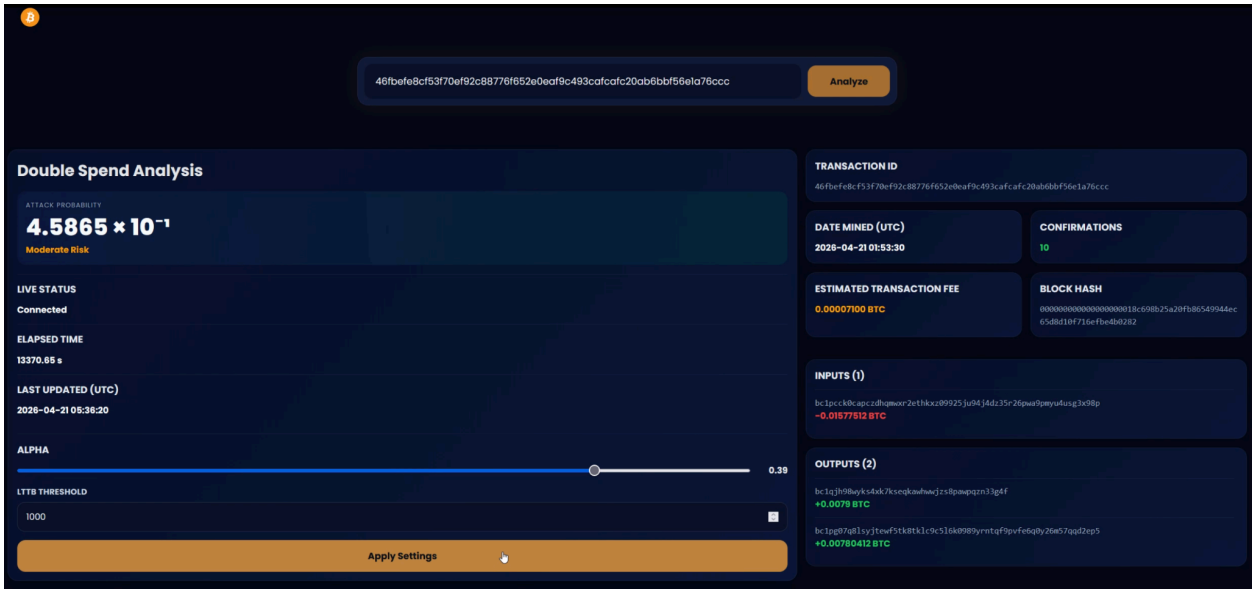


Figure 4. Final transaction result view



Figure 5. Historical and live probability graphs

6.2 Limitations

The project has several limitations. First, the probability output depends on model assumptions, especially the selected attacker hash-power value. A low probability in the graph does not guarantee that a transaction is safe in every real-world setting. Second, the system depends on access to a synchronized Bitcoin Core node. If the node is down, still syncing, or has RPC disabled, the website cannot provide current data. Third, the graphing features are focused on

representative visualizations rather than every possible acceptance policy. Fourth, unconfirmed transactions can only be successfully searched if it exists within our node's mempool. Finally, the deployment process still needs final packaging so a new user can start the services with minimal manual configuration.

6.3 Future Steps

- Document the exact Linux server deployment steps, including environment variables, RPC credentials, ports, and persistent volumes.
- Add automated tests for confirmed, mempool, old, and coinbase transaction cases.
- Add a small library of sample transaction IDs for demonstrations and regression testing.
- Improve the parameter controls so users can compare multiple attacker hash-power assumptions on the same graph.
- Add clearer explanatory text beside the probability graph so non-expert users understand that the output is model-based.
- Cache repeated transaction lookups to reduce load on the Bitcoin Core node.
- Add HTTPS and production logging before public deployment.

7. Summary and Conclusion

The Bitcoin Double-Spend Website demonstrates how a blockchain explorer can be extended into an educational risk-analysis tool. The project combines Bitcoin Core, a Python data processor, .NET middleware, and a Blazor user interface to show transaction details and double-spend probability graphs. During development, the team learned that the hardest parts of blockchain software are often edge cases and data interpretation rather than the first successful API call. Mempool transactions, coinbase transactions, old transactions, graph axes, and DTO alignment all required careful handling.

The project also reinforced the value of modular design. By separating the UI, middleware, data processor, and node, each layer has a clear purpose and can be debugged independently. The final application is a working prototype that meets the main educational goal, while future work should focus on deployment polish, automated testing, and clearer user-facing explanations of the probability model.

8. References

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] S. Neumayer, M. Varia, and I. Eyal, “An analysis of acceptance policies for blockchain transactions,” IACR Cryptology ePrint Archive, Rep. 2018/040, Jan. 2018. [Online]. Available: https://eprint.iacr.org/2018/040

Appendix A: User Manual

A.1 Minimum Requirements

The website is currently still in a development stage and is undeployed, so running the website can prove to be a bit more challenging than if it were deployed to the web. To run the website locally, you will need to be able to/have access to the following:

1. Have a Windows PC capable of running multiple docker containers.
2. Have an internet connection and are capable of connecting to UAA's ADAC server via GlobalProtect VPN.

The functionality of the current state of our website has not been verified on other platforms such as macOS and Linux, so Windows is recommended.

A.2 Starting the Program

Before starting the program, you will need to connect to the UAA ADAC server that the Bitcoin Core node is installed on. It is assumed that you have the proper tools and accounts to access this resource. First, you will need to connect to the UAA network via GlobalProtect VPN at the following address: [redacted]. When you have successfully entered your UAA credentials you will then need to open up PowerShell and type the following command while replacing <username> with your account info.

ssh -L 8332:127.0.0.1:8332 -L 28332:127.0.0.1:28332 <username>@[redacted]

After successfully logging in you may move onto the next steps but please note that the PowerShell window must remain open and logged in for the website to work.

The UI, .NET API and Python service are packaged into separate Docker containers. In the interest of time this will not cover how to set up Docker. When the three docker containers are loaded up into Docker, you can start running them in any order, when you start each container it is going to ask you to map the ports that the container uses. Please use the following:

Python_server: 8000

dotnet-api: 5037

Bitcoin-ui: 5002

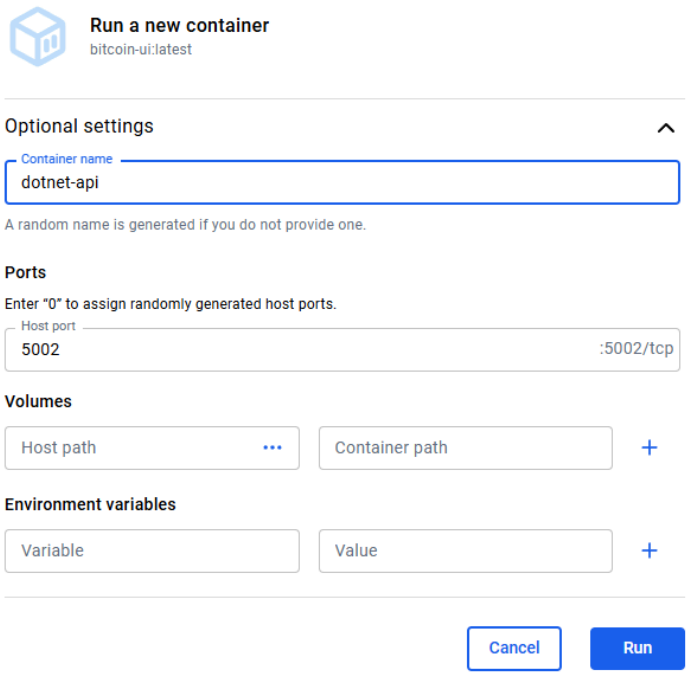


Figure 6. Example of port assignment when setting up the container.

After all the docker containers have been assigned ports and have successfully run you can now navigate to http://localhost:5002/ and the homepage of the website will appear. You are now ready to start searching transactions.

A.3 Searching for a Transaction

1. Copy a Bitcoin transaction ID
2. Paste the transaction ID into the search field on the home page or the search results page.
3. Submit the search.
4. Wait for the summary cards and graphs to load.
5. Review the confirmation count, block status, inputs, outputs, and probability card.
6. Watch the live graph if the transaction is recent enough for live updates to be useful.
7. Adjust model parameters and rerun the search.

A.4 Interpreting the Results

The transaction summary tells the user what happened on the blockchain. The probability graph tells the user how the modeled double-spend risk changes under the selected assumptions. A lower probability generally means the transaction has become harder to replace, usually because it has more confirmations. The graph should be interpreted as an educational model, not as financial advice.

A.5 Common Error Cases

Table 4. User Manual Error Cases

Symptom	**Likely cause**	**User action**
Transaction not found	Invalid transaction ID or node does not know the transaction.	Check the hash and try again.
No block time shown	The transaction may still be in the mempool.	Wait for confirmation or search for a confirmed transaction.
Coinbase input shown	The transaction creates the block subsidy and fees.	This is expected for coinbase transactions.
Graphs do not update	WebSocket connection failed or back end is unavailable.	Refresh the page or restart the back-end service.

Appendix B: Code Listing

Our project consists of 3 separate repositories for the front-end UI, .NET middleware and Python service that can be found at the links below.

Project repository folder:

https://bitbucket.org/doublespend/workspace/projects/DOUB

Python commits:

https://bitbucket.org/doublespend/dscap-py-worker/commits

Web API commits:

https://bitbucket.org/doublespend/dscap-api/commits

Front end UI commits:

https://bitbucket.org/doublespend/dscap-web/commits

Each group member was assigned to a part of the project. Caden and Lars' main responsibilities were the very important Python data processor and a Bitcoin Core node. Derek was mainly responsible for the middleware API and some of the front end. Zong was entirely responsible for the front end and user experience. While we had main areas of responsibility, group members were welcome to contribute to other parts of the project. Below is a table displaying the contribution splits of the major parts of the project.

Table 5. Source Code Files and Contributions

File or area	Purpose	Contributor split
Bitcoin Core configuration	Set up and configure the Bitcoin Core software onto the UAA ADAC server to use as our data source.	Caden
Python API endpoints	Defines HTTP and WebSocket routes, starts the Uvicorn/ASGI processor, and coordinates transaction updates.	Caden (90%), Lars (10%)
Python transaction summary	Calls Bitcoin Core RPC,	Caden

logic	handles coinbase/mempool cases, and builds the transaction summary JSON.	
Python probability logic	Calculates double-spend probability points for historical and live graphs.	Caden (10%), Lars (90%)
.NET Core API client/service files	Calls the Python service and maps responses to C# models.	Derek
C# DTO/model files	Define transaction summary, input/output, and probability point objects used by the UI.	Derek
SearchResults.Razor	Displays transaction results, summary cards, and graph components.	Derek (15%), Zong (85%)
Python Probability Graph Calculation	Precomputes all the static probability graphs to be sent to the front end	Lars
LiveProbabilityGraph.Razor	Displays live probability points received through the WebSocket stream.	Derek
Graph component files	Display five-hour and full-history probability graphs using ApexCharts.	Derek
CSS files	Define layout, cards, risk coloring, and responsive page behavior.	Derek (30%), Zong (70%)
Overall UI design	The display that the user interacts with. The design is largely focused on ease of use and accessibility.	Zong
Docker files / compose file	Package services for Linux server deployment.	Caden

Main dependencies: Bitcoin Core RPC/ZMQ, Python, Uvicorn/ASGI, .NET Core, Blazor, ApexCharts, and a modern web browser. The Python data processor depends on the Bitcoin Core node. The .NET middleware depends on the Python endpoints. The Blazor UI depends on the .NET API and the shared DTO definitions.

Appendix C: Weekly Meeting Logs

Table 6. Weekly Meeting Log

Date	Time (12 hr format)	Meeting topic
1/28	1:00 PM	Discussed project expectations with the client, including the goal of showing double-spend probability for a searched Bitcoin transaction. Initial ideas included a search bar, transaction summary page, homepage, and searched transaction page.
2/4	1:00 PM	Discussed website content for the homepage and search results page. Planned the minimum transaction details to display, including block or mempool status, transaction inputs and outputs, and an initial double-spend probability value.
2/11	1:00 PM	Finalized requirements for dynamically refreshing probability data. Discussed displaying time since transaction broadcast, number of blocks since the transaction, and updating the probability graph over time through the Python worker and front end.
2/18	1:00 PM	Reviewed progress on live probability updates and server access. Discussed client-side updates, frequent probability refreshes, alpha/hash-power assumptions, and connecting to the UAA ADAC server through SSH and VPN.
2/25	1:00 PM	Reviewed probability calculation details, including the double-spend equations, alpha parameter, and performance considerations such as saving Poisson parameters. Discussed graphing probability as a function of time since transaction broadcast.
3/18	1:00 PM	Showed client historical graph generation, live graph behavior, and Bitcoin Core RPC calls. Discussed using transaction history data from the Bitcoin node, graphing elapsed time, and applying LTTB to full-history data while rendering live data without LTTB.
3/25	1:00 PM	Tested transaction history and probability endpoints using real transaction IDs. Reviewed example transactions for regular confirmations, delayed confirmations, and poster/demo graphs.
4/1	1:00 PM	Reviewed website progress and graph layout. Discussed first-hour and five-hour graph views, example transactions with different

		confirmation patterns, and front-end display cases such as no confirmations, normal confirmations, and many confirmations.
4/8	1:00 PM	Reviewed front-end and back-end connection, including historical probability endpoints, graph data, and transaction lookup behavior. Discussed old transaction support, full-history performance, and whether to use LTTB or early stopping for large datasets.
4/15	12:40 PM	Reviewed current project progress, working demo preparation, and poster content. Discussed final demonstration examples, graph selection, and remaining cleanup such as removing raw calculations and improving graph presentation.